

Network Programming with Go

Concepts of Programming Languages

Sebastian Macke

Rosenheim Technical University

Last lecture

- Concurrent programming in general
- Coroutines
- Virtual threads of Go
- Mutex, Waitgroup
- Atomic operations
- Synchronization and messaging (channels)
- Multi-way concurrent control (select)
- Dining philosophers problem

About this Lecture

- Go was designed in the era of multicore, networked machines as light weight alternative to Java and C++.
- Hence we should take a look into the network capabilities of Go.
- Go provides a simple way to implement networking.
- Virtual Threads with non-blocking behavior should be ideal for receiving requests and waiting for database etc.
- So far, in all major languages, networking is not part of the language itself but provided via librarys.

What makes networking a programming concept?

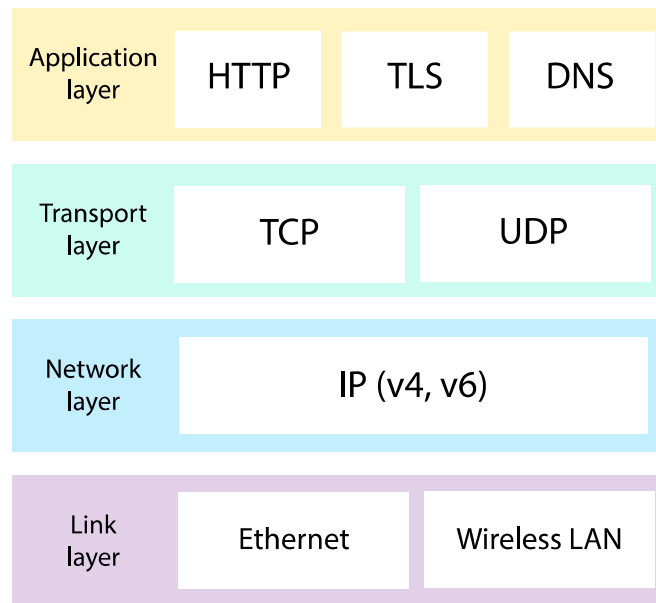
- Nowadays networking is closely linked to concurrency capabilities of a language.
- Let's take a look at Java:

Blog article

[Don't Crack Under Pressure: Java Microservices and the Battle for Stability Under High Load](https://blog.qaware.de/posts/java-service-queues/) (<https://blog.qaware.de/posts/java-service-queues/>)

4

The Internet protocol stack



- Many protocols power the Internet
- Each protocol operates at a different layer, building a stack of protocols
- The diagram is very simplified

HTTP

Hypertext Transfer Protocol

- Application Layer protocol based on TCP
- Mainly originally used to transfer Hypertext documents
- Classic operation mode: Client sends request, server sends response. Client sends next request

History:

- **HTTP 1.0:** 1989 developed at CERN by Tim Berners-Lee
- **HTTP 1.1:** Connection Pooling / Keep-Alive, Caching, Authentication, additional meta-information, ...
- **HTTP 2.0:** Binary-Protocol, Multiplexing, Encryption as Standard, Server-Push
- **HTTP 3.0:** Standardized 2023. Switch from TCP to UDP. Faster connection.

Example communication

Client sends the request to GET the content of the root path `"/index.html"`

```
GET /index.html HTTP/1.1  
Host: www.example.com
```

Server responds via

```
HTTP/1.1 200 OK  
Date: Mon, 23 May 2021 16:14:34 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 155  
Server: Apache  
  
<html>  
....
```

- Responds is a **text/html** document with Unicode UTF-8 encoding and has a size of 155 bytes

A Simple HTTP File Server

```
func main() {  
    port := flag.String("p", "8081", "port to serve on")  
    directory := flag.String("d", ".", "the directory of static file to host")  
    flag.Parse()  
  
    dirFs := os.DirFS(*directory)  
    http.Handle("/", http.FileServer(http.FS(dirFs)))  
    fmt.Printf("Starting server at port " + *port + "\n")  
    log.Fatal(http.ListenAndServe(":"+*port, nil))  
}
```

- File server for static content.
- Already optimized for maximum performance. No tweaking in any form necessary.

Python equivalent is still simpler

```
python -m http.server 8080
```

Transfer formats

There are different types of standardized transfer formats

- JSON (Text Format)
- XML (Text Format)
- Protobuf (Binary Format)

Truly portable data.

All can be transported via HTTP.

JSON - JavaScript Object Notation

```
{
  "coord": {
    "lon": -122.08,
    "lat": 37.39
  },
  "weather": [
    {
      "main": "Clear",
      "description": "clear sky",
    }
  ],
}
```

Types:

- bool
- floating point number
- strings
- arrays
- null

Marshalling and Unmarshalling

- Go offers a way to transform structs to the transfer formats and vice versa

```
type Weather struct {
    City      string `json:"city"` // The tag defines the exact name in the json.
    Sky       string `json:"sky"`
    Temperature int    `json:"temp"`
}

func main() {
    var weather Weather
    // Tags are accessible via reflection
    fmt.Println("Tag of first field entry", reflect.TypeOf(weather).Field(0).Tag)

    weatherJson := `{"city": "Rosenheim", "sky": "clear", "temp": 15}`
    err := json.Unmarshal([]byte(weatherJson), &weather)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("City: %s\nSky: %s\nTemperature: %d\n", weather.City, weather.Sky, weather.Temperature)
    s, _ := json.Marshal(weather)
    fmt.Println(string(s))
}
```

REST is a paradigm for application services

- Used mainly for Web Application Programming Interfaces
- Based on the HTTP Protocol
- Data is transferred via JSON or XML
- Stateless: The communication between server and client is stateless. The server "forgets" you after the request is done.
- Everything is a resource: A resource in the system should have only one logical URI
- Uniform interfaces based on HTTP. (GET , POST, PUT, DELETE). Errors are returned via HTTP status codes.

REST APIs with Go

Example: Weather API with Go: Short version

```
func main() {  
    response, err := http.Get("http://wttr.in/Rosenheim?format=j1")  
    if err != nil {  
        panic(err)  
    }  
    var forecast WeatherForecast  
    err = json.NewDecoder(response.Body).Decode(&forecast)  
    if err != nil {  
        panic(err)  
    }  
    fmt.Printf("Temperature in Rosenheim: %s°C\n", forecast.CurrentCondition[0].TempC)  
}
```

16

Exercise

Call the Github API with Go to receive the repositories with the most stars and the query "awesome"

api.github.com/search/repositories?sort=stars&order=desc&q=awesome (<https://api.github.com/search/repositories?sort=stars&order=desc&q=awesome>)

- Use

mholt.github.io/json-to-go/ (<https://mholt.github.io/json-to-go/>)

to instantly create a Go structure from an arbitrary JSON.

17

REST API with Go

```
func HandleNotFound(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusNotFound)
    fmt.Fprintf(w, "Nothing to see here\n")
}

func EchoParameters(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    data, _ := json.Marshal(r.URL.Query())
    w.Write(data)
}

func main() {
    http.HandleFunc("/", HandleNotFound)
    http.HandleFunc("/api/echoParams", EchoParameters)
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Each request is processed in its separate go routine.

To test the rest interface you can use tools such as

- curl, httpie, Postman, Your own Go Rest call code, Web Browser

Comparison with Java (Spring Boot)

```
@SpringBootApplication
@RestController
public class RestdemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(RestdemoApplication.class, args);
    }

    @GetMapping("/")
    public String index() {
        return "Greetings from Spring Boot!";
    }
}
```

- Frameworks such as Spring Boot offer a declarative approach via annotations

Thank you

Tags: [go](#), [programming](#), [concurrent](#), [go routines](#), [channels](#) ([#ZgotmplZ](#))

Sebastian Macke

Rosenheim Technical University

Sebastian.Macke@th-rosenheim.de (<mailto:Sebastian.Macke@th-rosenheim.de>)

<https://www.qaware.de> (<https://www.qaware.de>)

